Reachy_Local_08.4.2 Detailed Implementation Guide

Date: 2025-11-03 **Version**: 1.0 **Status**: Ready for Review

---

Executive Summary

This document provides detailed implementation specifications for completing Reachy_Local_08.4.2. Each section includes production-ready code with comprehensive logic explanations. **Do not implement yet** - this is for review and approval first.

Implementation Approach: Test-driven, incremental delivery with validation gates between phases.

Phase 1: Foundation Completion (Weeks 1-2)

1.1 API Client Enhancement with Retry Logic

File: `apps/web/api_client_enhanced.py`

Core Logic: Exponential backoff with jitter prevents thundering herd problem. Classifies errors as retryable (5xx, network) vs non-retryable (4xx). Idempotency keys prevent duplicate operations.

```python
# Retry decorator with exponential backoff
def with_retry(max_attempts=3, base_delay=1.0):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except requests.HTTPError as e:
                    if e.response.status_code < 500:
                        raise  # Don't retry client errors
                    if attempt == max_attempts - 1:
                        raise
                    delay = base_delay * (2 ** attempt) * random.uniform(0.5, 1.5)
                    time.sleep(delay)
        return wrapper
    return decorator

# Enhanced promote with idempotency
@with_retry(max_attempts=3)
```

```python
def promote_with_idempotency(video_id, dest_split, label=None):
    # Generate deterministic idempotency key
    key = hashlib.sha256(f"{video_id}:{dest_split}:{time.time()//60}".encode()).hexdigest()
    headers = _headers()
    headers["Idempotency-Key"] = key

    response = requests.post(
        f"{_base_url()}/promote",
        headers=headers,
        json={"video_id": video_id, "dest_split": dest_split, "label": label}
    )
    response.raise_for_status()
    return response.json()
```

Testing Strategy:

```python
def test_retry_on_500():
    with mock.patch('requests.post', side_effect=[
        requests.HTTPError(response=Mock(status_code=500)),
        Mock(json=lambda: {"status": "ok"})
    ]):
        result = promote_with_idempotency("vid123", "dataset_all")
        assert result["status"] == "ok"
```

1.2 WebSocket Integration

File: `apps/web/websocket_manager.py`

Core Logic: Maintains persistent connection with auto-reconnect. Messages queued for Streamlit polling. Event handlers dispatch to callbacks.

```python
class WebSocketManager:
    def __init__(self, url):
        self.sio = socketio.Client(reconnection=True, reconnection_delay=1)
        self.message_queue = queue.Queue()
        self._setup_handlers()

    def _setup_handlers(self):
        @self.sio.on('emotion_event')
        def on_emotion(data):
            self.message_queue.put(('emotion', data))

        @self.sio.on('promotion_complete')
        def on_promotion(data):
            self.message_queue.put(('promotion', data))
```

```python
    def connect(self):
        self.sio.connect(self.url)

    def get_messages(self, timeout=0.1):
        messages = []
        try:
            while True:
                messages.append(self.message_queue.get(timeout=timeout))
        except queue.Empty:
            pass
        return messages

# Streamlit integration
if 'ws_manager' not in st.session_state:
    st.session_state.ws_manager = WebSocketManager(gateway_url)
    st.session_state.ws_manager.connect()

# Poll for messages
for msg_type, data in st.session_state.ws_manager.get_messages():
    if msg_type == 'emotion':
        st.session_state.latest_emotion = data
```

1.3 Database Schema Additions

File: `alembic/versions/20251103_training_tables.py`

Core Logic: training_run tracks experiments with run_id. training_selection links videos to runs with constraints preventing duplicates. Triggers auto-update timestamps.

```sql
-- Training run table
CREATE TABLE training_run (
    run_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    status VARCHAR(50) CHECK (status IN ('pending','training','completed','failed')),
    strategy VARCHAR(100) NOT NULL,
    train_fraction NUMERIC(3,2) CHECK (train_fraction > 0 AND train_fraction < 1),
    test_fraction NUMERIC(3,2) CHECK (test_fraction > 0 AND test_fraction < 1),
    seed BIGINT,
    dataset_hash VARCHAR(64),
    mlflow_run_id VARCHAR(255),
    metrics JSONB,
    CONSTRAINT valid_fractions CHECK (train_fraction + test_fraction <= 1.0)
);
```

```sql
-- Selection junction table
CREATE TABLE training_selection (
    run_id UUID REFERENCES training_run(run_id) ON DELETE CASCADE,
    video_id UUID REFERENCES video(video_id) ON DELETE CASCADE,
    target_split VARCHAR(20) CHECK (target_split IN ('train','test')),
    selected_at TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (run_id, video_id, target_split)
);

-- Audit trail
CREATE TABLE promotion_log (
    id BIGSERIAL PRIMARY KEY,
    video_id UUID REFERENCES video(video_id),
    from_split VARCHAR(20),
    to_split VARCHAR(20),
    label VARCHAR(50),
    idempotency_key VARCHAR(255) UNIQUE,
    success BOOLEAN,
    error_message TEXT,
    promoted_at TIMESTAMPTZ DEFAULT NOW()
);
```

Stored Procedure for Balanced Sampling:

```sql
CREATE FUNCTION sample_balanced_dataset(
    p_run_id UUID,
    p_train_fraction NUMERIC,
    p_seed BIGINT
) RETURNS TABLE(video_id UUID, target_split VARCHAR) AS $$
BEGIN
    -- Set random seed for reproducibility
    PERFORM setseed(p_seed::DOUBLE PRECISION / 2147483647);

    -- Sample proportionally from each label class
    RETURN QUERY
    WITH per_class AS (
        SELECT
            v.video_id,
            v.label,
            CASE
                WHEN random() < p_train_fraction THEN 'train'
                ELSE 'test'
            END as split
        FROM video v
        WHERE v.split = 'dataset_all' AND v.label IS NOT NULL
    )
    SELECT pc.video_id, pc.split FROM per_class pc;
```

```
END;
$$ LANGUAGE plpgsql;
```

Phase 2: ML Pipeline (Weeks 3-5)

2.1 TAO Training Environment Setup

File: `trainer/tao/setup_environment.sh`

Core Logic: Docker compose orchestrates TAO 4.x (training) and TAO 5.3 (export) containers. Volume mounts provide access to dataset and models. GPU passthrough enables CUDA acceleration.

```bash
#!/bin/bash
# TAO Environment Setup Script

# Pull TAO containers
docker pull nvcr.io/nvidia/tao/tao-toolkit:4.0.0-tf2.11.0
docker pull nvcr.io/nvidia/tao/tao-toolkit:5.3.0-pyt

# Create docker-compose.yml
cat > docker-compose-tao.yml <<EOF
version: '3.8'
services:
  tao-train:
    image: nvcr.io/nvidia/tao/tao-toolkit:4.0.0-tf2.11.0
    runtime: nvidia
    volumes:
      - /media/project_data/reachy_emotion/videos:/workspace/data
      - ./trainer/tao/specs:/workspace/specs
      - ./trainer/tao/experiments:/workspace/experiments
    environment:
      - NVIDIA_VISIBLE_DEVICES=0
    command: tail -f /dev/null  # Keep running

  tao-export:
    image: nvcr.io/nvidia/tao/tao-toolkit:5.3.0-pyt
    runtime: nvidia
    volumes:
      - ./trainer/tao/experiments:/workspace/experiments
      - ./jetson/engines:/workspace/engines
    environment:
      - NVIDIA_VISIBLE_DEVICES=0
EOF

docker-compose -f docker-compose-tao.yml up -d
```

Training Specification:

File: `trainer/tao/specs/emotion_train_2cls.yaml`

```yaml
# EmotionNet 2-class training specification
model:
  arch: "resnet18"
  input_shape: [224, 224, 3]
  num_classes: 2
  pretrained_weights: "imagenet"
  freeze_blocks: [0, 1]  # Fine-tune last layers only

dataset:
  train_data_path: "/workspace/data/train"
  val_data_path: "/workspace/data/test"
  augmentation:
    enable: true
    random_flip: true
    random_crop: true
    color_jitter:
      brightness: 0.2
      contrast: 0.2
      saturation: 0.2

training:
  batch_size: 32
  epochs: 50
  optimizer: "adam"
  learning_rate: 0.001
  lr_schedule:
    type: "cosine"
    warmup_epochs: 5
  early_stopping:
    patience: 10
    metric: "val_f1"

validation:
  metrics: ["accuracy", "f1_macro", "confusion_matrix"]
  checkpoint_interval: 5
```

2.2 Dataset Preparation Script

File: `trainer/prepare_dataset.py`

Core Logic: Queries database for balanced sample using stored procedure. Copies files from dataset_all to train/test using run_id. Generates JSONL

manifests for TAO. Updates training_selection table.

```
import os
import shutil
import json
import hashlib
from pathlib import Path
import psycopg2
from typing import Dict, List

def prepare_training_dataset(
    run_id: str,
    train_fraction: float = 0.7,
    seed: int = None
) -> Dict[str, any]:
    """
    Prepare train/test splits for a training run.

    Logic:
    1. Create new training_run record in DB
    2. Call stored procedure for balanced sampling
    3. Copy files from dataset_all to train/test directories
    4. Generate manifests
    5. Update training_selection table
    6. Return metadata for MLflow logging
    """
    conn = psycopg2.connect(os.getenv('DATABASE_URL'))
    cur = conn.cursor()

    # Generate seed if not provided
    if seed is None:
        seed = int(hashlib.md5(run_id.encode()).hexdigest(), 16) % (2**31)

    # Create training run
    cur.execute("""
        INSERT INTO training_run (run_id, strategy, train_fraction, test_fraction, seed, sta
        VALUES (%s, 'balanced_random', %s, %s, %s, 'sampling')
        RETURNING dataset_hash
    """, (run_id, train_fraction, 1.0 - train_fraction, seed))

    # Sample dataset
    cur.execute("""
        SELECT video_id, target_split, file_path, label
        FROM sample_balanced_dataset(%s, %s, %s)
        JOIN video USING (video_id)
    """, (run_id, train_fraction, seed))
```

```
selections = cur.fetchall()

# Copy files and build manifests
train_manifest = []
test_manifest = []
base_path = Path("/media/project_data/reachy_emotion/videos")

for video_id, target_split, file_path, label in selections:
    src = base_path / file_path
    dst_dir = base_path / target_split / label if target_split == 'train' else base_path
    dst_dir.mkdir(parents=True, exist_ok=True)
    dst = dst_dir / src.name

    shutil.copy2(src, dst)

    # Add to manifest
    entry = {"video_id": str(video_id), "path": str(dst), "label": label if target_split
    if target_split == 'train':
        train_manifest.append(entry)
    else:
        test_manifest.append(entry)

    # Log selection
    cur.execute("""
        INSERT INTO training_selection (run_id, video_id, target_split)
        VALUES (%s, %s, %s)
    """, (run_id, video_id, target_split))

# Write manifests
manifest_dir = base_path / "manifests"
manifest_dir.mkdir(exist_ok=True)

with open(manifest_dir / f"{run_id}_train.jsonl", 'w') as f:
    for entry in train_manifest:
        f.write(json.dumps(entry) + '\n')

with open(manifest_dir / f"{run_id}_test.jsonl", 'w') as f:
    for entry in test_manifest:
        f.write(json.dumps(entry) + '\n')

# Update status
cur.execute("UPDATE training_run SET status = 'ready' WHERE run_id = %s", (run_id,))
conn.commit()

return {
```

```
        "run_id": run_id,
        "train_count": len(train_manifest),
        "test_count": len(test_manifest),
        "seed": seed
    }
```

2.3 MLflow Integration

File: `trainer/mlflow_tracker.py`

Core Logic: Wraps TAO training with MLflow tracking. Logs hyperparameters, metrics per epoch, artifacts (models, confusion matrices). Links dataset_hash for reproducibility.

```
import mlflow
import mlflow.tensorflow

def train_with_mlflow(run_id: str, config: dict):
    """
    Train model with MLflow tracking.

    Logic:
    1. Start MLflow run with run_id as name
    2. Log all hyperparameters from config
    3. Log training metrics each epoch
    4. Save model artifacts
    5. Log dataset hash for reproducibility
    """
    mlflow.set_experiment("emotion-classification")

    with mlflow.start_run(run_name=run_id):
        # Log config
        mlflow.log_params({
            "model_arch": config["model"]["arch"],
            "batch_size": config["training"]["batch_size"],
            "learning_rate": config["training"]["learning_rate"],
            "epochs": config["training"]["epochs"]
        })

        # Get dataset hash from DB
        conn = psycopg2.connect(os.getenv('DATABASE_URL'))
        cur = conn.cursor()
        cur.execute("SELECT dataset_hash FROM training_run WHERE run_id = %s", (run_id,))
        dataset_hash = cur.fetchone()[0]
        mlflow.log_param("dataset_hash", dataset_hash)
```

```python
# Train (TAO CLI wrapper)
result = subprocess.run([
    "tao", "emotionnet", "train",
    "-e", "/workspace/specs/emotion_train_2cls.yaml",
    "-r", f"/workspace/experiments/{run_id}",
    "-k", os.getenv("TAO_API_KEY")
], capture_output=True, text=True)

# Log metrics from TAO output
for line in result.stdout.split('\n'):
    if 'epoch' in line.lower():
        # Parse: "Epoch 10/50 - loss: 0.234 - accuracy: 0.856"
        parts = line.split('-')
        epoch = int(parts[0].split()[1].split('/')[0])
        metrics = dict(p.split(':') for p in parts[1:])
        for k, v in metrics.items():
            mlflow.log_metric(k.strip(), float(v.strip()), step=epoch)

# Log model artifacts
mlflow.log_artifacts(f"/workspace/experiments/{run_id}/weights")

return result.returncode == 0
```

Phase 3: Edge Deployment (Weeks 6-7)

3.1 DeepStream Pipeline Configuration

File: `jetson/deepstream/emotion_pipeline.txt`

Core Logic: GStreamer pipeline with nvinfer element for TensorRT inference. Preprocessing normalization, batching for efficiency, postprocessing for class probabilities.

```
[application]
enable-perf-measurement=1
perf-measurement-interval-sec=5

[source0]
enable=1
type=1  # V4L2 camera
camera-width=1920
camera-height=1080
camera-fps-n=30
camera-fps-d=1
```

```
[sink0]
enable=1
type=2  # File sink (for debugging)

[primary-gie]
enable=1
gpu-id=0
batch-size=1
bbox-border-color0=1;0;0;1
gie-unique-id=1
config-file=emotion_inference.txt
model-engine-file=../../engines/emotion_fp16.engine

[tracker]
enable=0  # No tracking needed for emotion

[nvds-analytics]
enable=0

[osd]
enable=1
border-width=2
text-size=12
text-color=1;1;1;1;
text-bg-color=0.3;0.3;0.3;1

[streammux]
gpu-id=0
batch-size=1
batched-push-timeout=40000
width=224
height=224
```

Inference Configuration:

File: `jetson/deepstream/emotion_inference.txt`

```
[property]
gpu-id=0
net-scale-factor=0.0039215697906911373  # 1/255
model-engine-file=../../engines/emotion_fp16.engine
labelfile-path=emotion_labels.txt
batch-size=1
network-mode=2  # FP16
num-detected-classes=2
interval=0  # Infer every frame
gie-unique-id=1
```

```
output-blob-names=dense_2/Softmax

[class-attrs-all]
pre-cluster-threshold=0.5  # Confidence threshold
```

3.2 TensorRT Export Script

File: `trainer/tao/export_to_tensorrt.sh`

Core Logic: Uses TAO 5.3 to convert trained model to TensorRT engine. FP16 precision for speed/memory balance. INT8 calibration optional for further optimization.

```bash
#!/bin/bash
# Export EmotionNet model to TensorRT

RUN_ID=$1
MODEL_PATH="/workspace/experiments/${RUN_ID}/weights/model.hdf5"
OUTPUT_PATH="/workspace/engines/emotion_${RUN_ID}_fp16.engine"

# Export with TAO 5.3
docker-compose -f docker-compose-tao.yml exec tao-export tao emotionnet export \
  -m ${MODEL_PATH} \
  -k $(cat /workspace/.tao_key) \
  -o ${OUTPUT_PATH} \
  --data_type fp16 \
  --batch_size 1 \
  --input_dims 3,224,224

# Verify engine
docker-compose -f docker-compose-tao.yml exec tao-export trtexec \
  --loadEngine=${OUTPUT_PATH} \
  --shapes=input:1x3x224x224 \
  --verbose

echo "Engine exported to ${OUTPUT_PATH}"
```

3.3 Jetson WebSocket Client

File: `jetson/emotion_client.py`

Core Logic: Connects to Ubuntu 2 gateway via WebSocket. Sends emotion events with device_id, timestamp, inference metrics. Receives cues for gestures/TTS.

```python
import socketio
import time
from datetime import datetime

class EmotionClient:
    def __init__(self, gateway_url, device_id):
        self.sio = socketio.Client(reconnection=True)
        self.device_id = device_id
        self.gateway_url = gateway_url
        self._setup_handlers()

    def _setup_handlers(self):
        @self.sio.on('connect')
        def on_connect():
            print(f"Connected to {self.gateway_url}")
            self.sio.emit('register', {
                'device_id': self.device_id,
                'device_type': 'jetson'
            })

        @self.sio.on('cue')
        def on_cue(data):
            # Handle gesture or TTS cue from server
            if data['type'] == 'gesture':
                self.trigger_gesture(data['gesture_id'])
            elif data['type'] == 'tts':
                self.speak(data['text'])

    def send_emotion(self, emotion: str, confidence: float, inference_ms: float):
        """Send emotion detection event to server."""
        payload = {
            "device_id": self.device_id,
            "ts": datetime.utcnow().isoformat() + 'Z',
            "emotion": emotion,
            "confidence": float(confidence),
            "inference_ms": float(inference_ms),
            "window": {"fps": 30, "size_s": 1.0, "hop_s": 0.5}
        }
        self.sio.emit('emotion_event', payload)

    def connect(self):
        self.sio.connect(self.gateway_url)

    def run(self):
        self.connect()
        self.sio.wait()
```

Phase 4: n8n Orchestration (Weeks 8-9)

4.1 Ingest Agent Workflow

File: `n8n/workflows/01_ingest_agent.json`

Core Logic: Triggered by webhook or file upload. Computes SHA256 hash, extracts metadata with ffprobe, generates thumbnail, writes to DB, emits completion event.

```
{
  "name": "Ingest Agent",
  "nodes": [
    {
      "name": "Webhook Trigger",
      "type": "n8n-nodes-base.webhook",
      "parameters": {
        "path": "ingest",
        "method": "POST"
      }
    },
    {
      "name": "Compute Hash",
      "type": "n8n-nodes-base.executeCommand",
      "parameters": {
        "command": "sha256sum {{$json.file_path}}"
      }
    },
    {
      "name": "Extract Metadata",
      "type": "n8n-nodes-base.executeCommand",
      "parameters": {
        "command": "ffprobe -v quiet -print_format json -show_format -show_streams {{$json.f
      }
    },
    {
      "name": "Generate Thumbnail",
      "type": "n8n-nodes-base.executeCommand",
      "parameters": {
        "command": "ffmpeg -i {{$json.file_path}} -vframes 1 -vf scale=320:-1 /thumbs/{{$jso
      }
    },
    {
      "name": "Write to Database",
      "type": "n8n-nodes-base.postgres",
```

```
      "parameters": {
        "operation": "insert",
        "table": "video",
        "columns": "video_id,file_path,sha256,duration_sec,width,height,size_bytes"
      }
    },
    {
      "name": "Emit Event",
      "type": "n8n-nodes-base.webhook",
      "parameters": {
        "method": "POST",
        "url": "http://ubuntu2:8000/api/events/ingest_complete"
      }
    }
  ]
}
```

4.2 Training Orchestrator Workflow

File: `n8n/workflows/05_training_orchestrator.json`

Core Logic: Polls database for dataset readiness (balanced, sufficient samples). Triggers dataset preparation, launches TAO training, monitors progress, validates gates, exports TensorRT engine.

```
{
  "name": "Training Orchestrator",
  "nodes": [
    {
      "name": "Schedule Check",
      "type": "n8n-nodes-base.cron",
      "parameters": {
        "triggerTimes": "0 */6 * * *"  # Every 6 hours
      }
    },
    {
      "name": "Check Dataset Readiness",
      "type": "n8n-nodes-base.postgres",
      "parameters": {
        "query": "SELECT * FROM is_dataset_balanced(100, 1.5)"
      }
    },
    {
      "name": "Decision: Ready?",
      "type": "n8n-nodes-base.if",
```

```
      "parameters": {
        "conditions": {
          "boolean": [{
            "value1": "={{$json.balanced}}",
            "value2": true
          }]
        }
      }
    },
    {
      "name": "Prepare Dataset",
      "type": "n8n-nodes-base.executeCommand",
      "parameters": {
        "command": "python /workspace/trainer/prepare_dataset.py --run-id {{$json.run_id}}"
      }
    },
    {
      "name": "Train with TAO",
      "type": "n8n-nodes-base.executeCommand",
      "parameters": {
        "command": "docker-compose -f docker-compose-tao.yml exec tao-train tao emotionnet t
      }
    },
    {
      "name": "Validate Gate A",
      "type": "n8n-nodes-base.postgres",
      "parameters": {
        "query": "SELECT metrics->>'f1_macro' as f1 FROM training_run WHERE run_id = '{{$jso
      }
    },
    {
      "name": "Export to TensorRT",
      "type": "n8n-nodes-base.executeCommand",
      "parameters": {
        "command": "/workspace/trainer/tao/export_to_tensorrt.sh {{$json.run_id}}"
      }
    }
  ]
}
```

Phase 5: Production Hardening (Weeks 10-12)

5.1 Grafana Dashboard Configuration

File: `monitoring/dashboards/system_health.json`

Key Metrics: - API latency (p50, p95, p99) - Error rate by endpoint - Database connection pool utilization - Video processing queue depth - Training run status - Jetson inference FPS and latency - GPU utilization and temperature

Alert Rules:

```yaml
groups:
  - name: reachy_alerts
    rules:
      - alert: HighErrorRate
        expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.01
        annotations:
          summary: "High error rate detected"

      - alert: SlowInference
        expr: histogram_quantile(0.95, inference_latency_seconds) > 0.25
        annotations:
          summary: "Inference latency above 250ms (p95)"

      - alert: LowAccuracy
        expr: model_accuracy < 0.80
        annotations:
          summary: "Model accuracy below 80%"
```

5.2 JWT Authentication

File: `apps/api/app/auth.py`

Core Logic: Middleware validates Bearer tokens. Uses RS256 asymmetric encryption. Tokens expire after 15 minutes. Refresh tokens valid for 7 days.

```python
from fastapi import HTTPException, Security
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt
from datetime import datetime, timedelta

security = HTTPBearer()

def create_access_token(user_id: str) -> str:
    """Create JWT access token (15 min expiry)."""
    payload = {
        "sub": user_id,
        "exp": datetime.utcnow() + timedelta(minutes=15),
        "iat": datetime.utcnow(),
        "type": "access"
    }
```

```python
    return jwt.encode(payload, PRIVATE_KEY, algorithm="RS256")

def verify_token(credentials: HTTPAuthorizationCredentials = Security(security)) -> str:
    """Verify JWT token and return user_id."""
    try:
        payload = jwt.decode(
            credentials.credentials,
            PUBLIC_KEY,
            algorithms=["RS256"]
        )
        return payload["sub"]
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Token expired")
    except jwt.JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")

# Apply to routes
@router.post("/promote", dependencies=[Depends(verify_token)])
async def promote_video(...):
    pass
```

Testing Strategy

Unit Tests

- API client retry logic with mocked failures
- Database stored procedures with test fixtures
- WebSocket message handling

Integration Tests

- End-to-end promotion workflow: temp -> dataset_all
- Training run creation and sampling
- WebSocket event delivery

System Tests

- Load testing with locust (1000 req/s)
- Failover testing (kill services, verify recovery)
- Data integrity (checksums, manifest validation)

Deployment Checklist

- ☐ Run database migrations

- ☐ Generate SSL certificates for mTLS
- ☐ Configure environment variables in .env
- ☐ Build and tag Docker images
- ☐ Deploy to staging environment
- ☐ Run smoke tests
- ☐ Deploy to production
- ☐ Monitor metrics for 24 hours
- ☐ Document rollback procedure

End of Implementation Guide

Next Steps: Review this plan, provide feedback, approve for implementation.